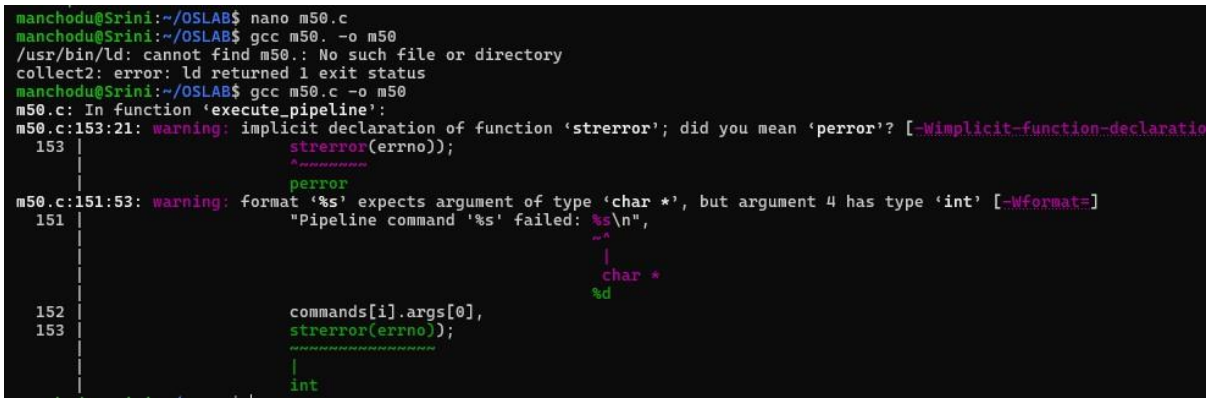

manchodu@Srini:~/OSLAB$ nano m50.c
manchodu@Srini:~/OSLAB$ gcc m50. -o m50
/usr/bin/ld: cannot find m50.: No such file or directory
collect2: error: ld returned 1 exit status
manchodu@Srini:~/OSLAB$ gcc m50.c -o m50
m50.c: In function ‘execute_pipeline’:
m50.c:153:21: warning: implicit declaration of function ‘strerror’; did you mean ‘perror’? [-Wimplicit-function-declaratio
153 | strerror(errno));
| ^~~~~~~~
| perror
m50.c:151:53: warning: format ‘%s’ expects argument of type ‘char *’, but argument 4 has type ‘int’ [-Wformat=]
151 | "Pipeline command '%s' failed: %s\n",
| ~^
| |
| char *
| %d
152 | commands[i].args[0],
153 | strerror(errno));
| ~~~~~~~~~~~~~~~~
| |
| int

```c
/*
 * pipeline_stability.c
 *
 * Execute Large Pipelines
 * Launch Multiple Jobs
 * Measure Stability
 * Monitor Resources
 * Detect Failures
 * Analyze Performance
 *
 * Compile:
 * gcc -Wall -Wextra pipeline_stability.c -o pipeline_stability
 *
 * Run:
 * ./pipeline_stability
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/resource.h>
#include <signal.h>
#include <time.h>
#include <errno.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 10
#define MAX_JOBS 5

typedef struct
{
    char *argv[MAX_ARGS];
} Command;

typedef struct
{
    pid_t pid;
    int status;
    int completed;
} Job;

void print_resource_usage()
{
    struct rusage usage;

    if (getrusage(RUSAGE_CHILDREN, &usage) == -1)
    {
        perror("getrusage");
        return;
    }

    printf("\n========== RESOURCE USAGE ==========\n");

    printf("User CPU Time   : %ld.%06ld sec\n",
           usage.ru_utime.tv_sec,
           usage.ru_utime.tv_usec);

    printf("System CPU Time : %ld.%06ld sec\n",
           usage.ru_stime.tv_sec,
           usage.ru_stime.tv_usec);

    printf("Maximum RSS     : %ld KB\n",
           usage.ru_maxrss);

    printf("Voluntary CS    : %ld\n",
           usage.ru_nvcsw);

    printf("Involuntary CS  : %ld\n",
           usage.ru_nivcsw);
}

int execute_pipeline(Command commands[], int count)
{
    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];

    if (count <= 0 || count > MAX_COMMANDS)
    {
        fprintf(stderr, "Invalid pipeline size.\n");
        return -1;
    }

    printf("\n================================================\n");
    printf("          LARGE PIPELINE EXECUTION\n");
    printf("================================================\n");

    for (int i = 0; i < count - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");

            for (int j = 0; j < i; j++)
            {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }

            return -1;
        }
    }

    for (int i = 0; i < count; i++)
    {
        pids[i] = fork();

        if (pids[i] == -1)
        {
            perror("fork");

            for (int j = 0; j < count - 1; j++)
            {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }

            return -1;
        }

        if (pids[i] == 0)
        {
            if (i > 0)
            {
                if (dup2(pipes[i - 1][0], STDIN_FILENO) == -1)
                {
                    perror("dup2 stdin");
                    exit(EXIT_FAILURE);
                }
            }

            if (i < count - 1)
            {
                if (dup2(pipes[i][1], STDOUT_FILENO) == -1)
                {
                    perror("dup2 stdout");
                    exit(EXIT_FAILURE);
                }
            }

            for (int j = 0; j < count - 1; j++)
            {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }

            execvp(commands[i].argv[0], commands[i].argv);

            fprintf(stderr,
                    "Pipeline command '%s' failed: %s\n",
                    commands[i].argv[0],
                    strerror(errno));

            exit(127);
        }
    }

    for (int i = 0; i < count - 1; i++)
    {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    int final_status = 0;

    for (int i = 0; i < count; i++)
    {
        int status;

        if (waitpid(pids[i], &status, 0) == -1)
        {
            perror("waitpid");
```

```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m51.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m51.c -o m51
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m51
==========================================
          PIPELINE MONITORING SUITE
==========================================

------------------------------------------
Performance Test 1
------------------------------------------
3439
Result: SUCCESS
Execution time: 0.008749 seconds

------------------------------------------
Performance Test 2
------------------------------------------
3439
Result: SUCCESS
Execution time: 0.003218 seconds

------------------------------------------
Performance Test 3
------------------------------------------
3439
Result: SUCCESS
Execution time: 0.003274 seconds

------------------------------------------
Performance Test 4
------------------------------------------
3439
Result: SUCCESS
Execution time: 0.002546 seconds

------------------------------------------
Performance Test 5
------------------------------------------
3439
Result: SUCCESS
Execution time: 0.002728 seconds

==========================================
              RESOURCE REPORT
==========================================
User CPU time   : 0.018495 sec
System CPU time : 0.004068 sec
Maximum RSS     : 2164 KB
Voluntary CS    : 24
Involuntary CS  : 0

All performance tests completed.
```

```
anirudh@LAPTOP-H0V5A2PN:~/OSLAB$ cat m51.c
/*
 * pipeline_monitor.c
 *
 * Monitor Resources
 * Detect Failures
 * Analyze Performance
 * Repeated Pipeline Testing
 *
 * Compile:
 * gcc -Wall -Wextra pipeline_monitor.c -o pipeline_monitor
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/resource.h>
#include <time.h>
#include <errno.h>
#include <string.h>

#define TEST_RUNS 5

void run_test(int test_number)
{
    int pipe1[2];
    int pipe2[2];

    pid_t p1;
    pid_t p2;
    pid_t p3;

    struct timespec start;
    struct timespec end;

    printf("\n----------------------------------------------\n");
    printf("Performance Test %d\n", test_number);
    printf("----------------------------------------------\n");

    if (pipe(pipe1) == -1)
    {
        perror("pipe1");
        return;
    }

    if (pipe(pipe2) == -1)
    {
        perror("pipe2");
        close(pipe1[0]);
        close(pipe1[1]);
        return;
    }

    clock_gettime(CLOCK_MONOTONIC, &start);

    p1 = fork();

    if (p1 == -1)
    {
        perror("fork");
        return;
    }

    if (p1 == 0)
    {
        dup2(pipe1[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);

        close(pipe2[0]);
        close(pipe2[1]);

        execlp("seq", "seq", "1", "10000", NULL);

        perror("exec seq");
        exit(127);
    }

    p2 = fork();

    if (p2 == -1)
    {
        perror("fork");
        return;
    }

    if (p2 == 0)
    {
```

```
        dup2(pipe1[0], STDIN_FILENO);
        dup2(pipe2[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("grep", "grep", "5", NULL);

        perror("exec grep");
        exit(127);
    }

    p3 = fork();

    if (p3 == -1)
    {
        perror("fork");
        return;
    }

    if (p3 == 0)
    {
        dup2(pipe2[0], STDIN_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("wc", "wc", "-l", NULL);

        perror("exec wc");
        exit(127);
    }

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    int failed = 0;

    pid_t pids[3] = {p1, p2, p3};

    for (int i = 0; i < 3; i++)
    {
        int status;

        if (waitpid(pids[i], &status, 0) == -1)
        {
            perror("waitpid");
            failed = 1;
            continue;
        }

        if (WIFEXITED(status))
        {
            int code = WEXITSTATUS(status);

            if (code != 0)
            {
                printf("PID %d failed with exit code %d\n",
                       pids[i],
                       code);

                failed = 1;
            }
        }
        else if (WIFSIGNALED(status))
        {
            printf("PID %d terminated by signal %d\n",
                   pids[i],
                   WTERMSIG(status));

            failed = 1;
        }
    }

    clock_gettime(CLOCK_MONOTONIC, &end);

    double elapsed =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1000000000.0;

    if (failed)
        printf("Result: FAILURE\n");
    else
        printf("Result: SUCCESS\n");
```

```
    else
        printf("Result: SUCCESS\n");

    printf("Execution time: %.6f seconds\n",
           elapsed);
}

int main()
{
    struct rusage usage;

    printf("================================================\n");
    printf("          PIPELINE MONITORING SUITE\n");
    printf("================================================\n");

    for (int i = 1; i <= TEST_RUNS; i++)
    {
        run_test(i);
    }

    if (getrusage(RUSAGE_CHILDREN, &usage) == 0)
    {
        printf("\n================================================\n");
        printf("               RESOURCE REPORT\n");
        printf("================================================\n");

        printf("User CPU time   : %ld.%06ld sec\n",
               usage.ru_utime.tv_sec,
               usage.ru_utime.tv_usec);

        printf("System CPU time : %ld.%06ld sec\n",
               usage.ru_stime.tv_sec,
               usage.ru_stime.tv_usec);

        printf("Maximum RSS     : %ld KB\n",
               usage.ru_maxrss);

        printf("Voluntary CS    : %ld\n",
               usage.ru_nvcsw);

        printf("Involuntary CS  : %ld\n",
               usage.ru_nivcsw);
    }

    printf("\nAll performance tests completed.\n");

    return 0;
}
anirudh@LAPTOP-H9VE42PN:~/OSLAB$
```